Transformers, tokens and embeddings: detailed walkthrough

Big picture

Aim. Give participants a working feel for how a language model reads biological sequences, before the notebooks that use real cancer data. Nothing here uses the CCLE dataset. Participants run a small protein language model (ESM-2, 8 million parameters) on a CPU, see how text, protein and DNA are cut into **tokens**, play the "fill in the hidden amino acid" game the model was trained on, look at its **attention**, and finish with a **control experiment**. About 90 minutes, no GPU, no maths.

The model. `facebook/esm2_t6_8M_UR50D` from Hugging Face: 6 layers, 20 attention heads per layer, about 35 MB. It was trained only on protein sequences by hiding residues and asking it to guess them.

The test protein. Human ubiquitin, 76 amino acids, small and very well studied. Every exercise uses the same sequence, so results can be compared between participants.

What participants do. Short TODOs (counting tokens, a dictionary of tokens per character, one lookup of a probability, a loop over positions, one line for attention distance, a scramble), each followed by a check cell that says whether the answer is right. Hints are hidden under a triangle. Each part follows the same rhythm: demo (5 min), try it yourself (8-10 min), debrief (3-5 min). Before each result there is a "your guess" text cell.

What they should leave with. (1) A token is the piece a model reads, and how a sequence is cut is a design choice. (2) A masked language model gives a probability for every possible answer. (3) Attention shows where the model looks, not why it answered. (4) A control such as a scrambled sequence tells you whether a result means something. These are the same habits the later notebooks use to judge mutations.

Part	Minutes	What happens
0 Setup	10	Install, load the model, define ubiquitin
1 Tokens	20	Text, protein and DNA tokenizers, then DNABERT-2 and GPT-2 (Part 1b)
2 Masked prediction	20	Probability of the true residue at each position

Part	Minutes	What happens
3 Attention	20	6 × 20 heads, near and far
4 Scramble test	10	Native against scrambled ubiquitin
5 Open time	10	Challenges and questions

Part 0: Setup (about 10 min)

Aim. Get the model running on a plain CPU.

What the cells do.

1. Install `transformers` and `matplotlib`, import NumPy, PyTorch and the ESM classes, and pick `cuda` if a GPU exists, otherwise `cpu`. Output: the library versions and the device (in the real run: transformers 5.22.1, torch 2.14.1, device cpu).
2. Load the tokenizer and `EsmForMaskedLM`. The model is loaded with eager attention so that attention weights can be read in Part 3, and a plain load is the fallback. It prints the size: 7.5 million parameters.
3. Define `ubq`, the 76-residue ubiquitin, with an assert on its length.
4. A text cell asks for a one-sentence guess about what a model trained only on protein sequences could learn.

What can go wrong. Downloads can be slow when 20 people start at once (the instructor note says to send Part 0 the day before). An unauthenticated-request warning from Hugging Face is harmless; an `HF_TOKEN` in Colab Secrets removes rate-limit trouble. After the install cell a `ModuleNotFoundError` is solved with Runtime > Restart session.

Part 1: Tokens (about 20 min)

Aim. Show that a model never reads letters, only numbered tokens, and that different kinds of sequence are cut differently.

Demos.

- **English text** with the `bert-base-uncased` tokenizer: `Bioinformatics uses transformers.` becomes `bio`, `##in`, `##form`, `##atics`, `uses`, `transformers`, `.` Rare words are split into pieces.

- **Protein** with the ESM tokenizer: one token per amino acid plus a start token `<cls>` and an end token `<eos>`. First tokens `<cls> M Q I F V K T`, IDs `0, 20, 16, 12, 18, 7, 15, 11`.
- **DNA** has no standard. The `kmers(seq, k, step)` helper cuts a sequence into chunks: `ATGGCGTACGTT` gives overlapping 3-mers `ATG, TGG, GGC, ...` (step 1) or non-overlapping `ATG, GCG, TAC, GTT` (step 3).

Exercise 1 (TODO).

- **1a** number of ESM tokens for ubiquitin: `len(enc["input_ids"])`, expected **78** (76 + 2 special tokens).
- **1b** a 30-base DNA string cut into 6-mers: **25** tokens overlapping (step 1), **5** non-overlapping (step 6).
- **1c** the ID of the mask token: `tok.mask_token_id`. It is needed in Part 2.

Check cell. Asserts the three answers and prints `Correct! 1a, 1b, 1c all pass.` In the real run it passed.

Debrief. Where do the two extra protein tokens come from? The same 30 bases gave 25 or 5 tokens: which chunking lets a fixed context length cover more genome, and what is lost? Some DNA models use one token per base; what would that need?

Part 1b: tokenizers you will meet in practice (about 10 min)

Aim. Two tokenizers that matter for genomics work.

1. **DNABERT-2** uses byte-pair encoding learned from genome sequence, so tokens are 2, 4 or 7 bases and repeated motifs become single tokens. Only the tokenizer files are downloaded. If the download fails the notebook prints a note and continues.
2. **GPT-2**, a general text tokenizer built from web text. Gene names, rsIDs and variant names are cut into arbitrary pieces.

Real numbers. The 64-base DNA demo gave **15 BPE tokens**, **4.27 bases per token**, against 1.08 for overlapping 6-mers and 6.4 for non-overlapping 6-mers. GPT-2 gave: `BRCA1` 4 tokens (`B RC A 1`), `BRCA2` 4, `brca1` 3 (`br ca 1`), `rs80357906` 5, `NM_007294.4` 6, `p.Cys61Gly` 7, `c.5266dupC` 7, the full HGVS string 19, `C61G` 3.

Exercise 1b (TODO). 1d compute tokens per character for every string in `bio_strings` and store it in a dictionary `tokens_per_char`; **1e** the string with the highest value, `most_fragmented`. Answer: **BRCA1** (0.80 tokens per character). The check asserts a dictionary, one entry per string, values between 0 and 1.

Debrief. Which tokenization covers the most DNA, and which keeps single-base detail? A text model cannot look up `rs80357906`; it can only recall associations between fragments, which invites confident but wrong answers (this returns in the last notebook). The same variant can be written `C61G`, `p.Cys61Gly` or `p.(Cys61Gly)`, so names should be normalised in code before they reach a model.

Part 2: Masked prediction, fill in the blank (about 20 min)

Aim. Show that the model answers with a probability for each of the 20 amino acids, not with one letter.

`masked_predictions(seq, pos, top=5)`. It tokenizes the sequence, replaces the residue at 0-based position `pos` with the mask token (index `pos + 1`, because position 0 is `<cls>`), runs the model, applies softmax to turn scores into probabilities that sum to 1, and returns the top guesses and all probabilities.

Real numbers. Position 3 (true F): the model spreads probability over E 0.162, T 0.161, K 0.160. Position 45 (true A): N 0.573, D 0.120, G 0.077, so the true residue is not first.

Exercise 2a (TODO). Pick a position, write a guess, then look up the probability of the true residue: `probs[tok.convert_tokens_to_ids(ubq[my_pos])]` as a plain float. For position 45 the answer is **0.023**. Check: probability between 0 and 1; prints `OK, probability = 0.023`.

Exercise 2b (TODO). Repeat for all 76 positions, one line inside a loop, and collect `p_true`. The check asserts 76 floats, then draws a bar plot (orange for hydrophobic residues, blue for the rest) with a dashed line at the mean. The true mean over the 76 positions in the real run, from the native-sequence comparison in Part 4, is **0.155**.

Debrief. Which positions are easy, which hard, and is there a pattern by residue type? The model never saw a 3D structure; what could it have learned? A low probability is not wrong biology: positions that vary between species are legitimately hard to predict.

Part 3: Attention (about 20 min)

Aim. Look inside the model, and learn what attention can and cannot say.

Concepts. One **head** is one learned way of looking; a **layer** has many heads; ESM-2 8M has 6 layers × 20 heads = 120 heads. The attention weight is between 0 and 1.

Cells.

1. Run the model with `output_attentions=True`, stack the layers into an array of shape layers × heads × L × L, drop the `<cls>` and `<eos>` rows and columns, and renormalise each row to sum to 1. Output: shape `(6, 20, 76, 76)`. An assert explains what to do if attentions come back as `None` (reload the model, which asks for eager attention).
2. `plot_head(layer, head)` and `plot_layer(layer)` draw heatmaps: rows are the residue that looks, columns the residue looked at.

Exercise 3 (TODO). Average attention distance per head: build `dist = |i - j|` for every pair of residues, then `mean_dist = (A * dist).sum(-1).mean(-1)`, shape (6, 20). Small means the head looks at neighbours, large means distant residues. Then `np.unravel_index(mean_dist.argmin(), ...)` and `argmax` find the most local and the most long-range head.

Real result. Most local head: layer 2, head 8 (mean distance 1.1). Most long-range head: layer 4, head 10 (mean distance 35.3).

Debrief and the caution to say out loud. The local head has a bright diagonal; the long-range head is blotchy. Distant residues can touch in 3D, so a long-range head is interesting, but the notebook did not test whether it found contacts. Attention shows where the model looks, not why it answered, and is a hint for exploring, not evidence. An optional `bertviz` cell gives an interactive view; it can be skipped.

Part 4: The scramble test (about 10 min)

Aim. Teach the idea of a negative control.

Cells. `p_true_all(seq)` is a faster version of the Part 2 loop: it builds all 76 masked copies of the sequence as one batch. A sanity assert checks that it matches the slow loop from Exercise 2b (printed: `batch version matches your loop from 2b`).

Exercise 4 (TODO). A fixed seed shuffles the 76 letters (same composition, random order), then both sequences are scored with `p_true_all`. The check asserts same letters, different order.

Real result. Mean probability of the true residue **0.155 for native ubiquitin against 0.064 for the scrambled sequence**. The scrambled protein is clearly worse: the model has learned something about real proteins, not only about amino-acid composition.

Debrief. This was one scramble of one protein; more scrambles, more proteins or another control would make it more convincing. The next notebooks use exactly this kind of score to judge mutations: a mutation that makes the model less happy with the sequence may

be damaging.

Part 5: Open time, challenges and problems (about 10 min)

Challenge A pastes a protein of up to 300 residues from UniProt and plots where the model is confident (default: human insulin B chain, mean probability 0.054 in the real run). **B** a bigger model (`esm2_t12_35M`). **C** mask several residues at once. **D** a preview of mutations: hide position 45 and compare the probability of the true residue with a different residue. **E** overlapping 4-mers and how many distinct ones exist ($4^4 = 256$).

Common problems.

- *Model download slow or fails (429, timeout):* wait a minute, or add an `HF_TOKEN` in Colab Secrets.
- *A check cell fails with a `...` message:* the placeholder was not replaced; fix the TODO cell and re-run it and its check.
- *`NameError`:* a cell above was not run; use Runtime > Run before.
- *Attentions are None:* re-run the model-loading cell in Part 0.
- *Numbers differ slightly from a neighbour:* small CPU and GPU differences are normal.

Instructor note. The first Colab run passed every check with no errors. The instructor-notes cell of the notebook still has blanks under "Values to verify"; the numbers above are the ones to write there.

Data retrieval from papers and the CCLE dataset: detailed walkthrough

Nov 12, 2026 · @Amir

A cell-by-cell explanation of the notebooks in this guide that use the CCLE data (data retrieval, AlphaGenome and folding): what each step does, what goes in, what comes out, what participants do, and how to tell it worked. One tab per notebook.

Data retrieval in one paragraph

Most of the data a biologist needs is not in a tidy database. It sits in the text of papers and in their supplementary Excel files. An AI assistant can write the code to pull it out, but AI code usually runs, so wrong data looks right. This notebook has participants get real data from real papers with AI-written code, and it trains one habit: ask clearly, then check before you trust. By the end they have saved `ccle_multiomics_panel.csv`, the table that the AlphaGenome, folding and final notebooks use.

Two papers, and why

The text and the data live in different places, so two papers are used.

Paper	What we take from it	How it is reached
Nusinow et al. 2020, *Quantitative Proteomics of the Cancer Cell Line Encyclopedia*, Cell	The protein levels of about 375 cell lines: Table S1 (the cell lines) and Table S2 (the proteins)	Direct downloads from the authors' lab site (`gygi.hms.harvard.edu/data/ccle/`). Europe PMC does not serve this paper's text: it answered HTTP 500 in the first Colab runs
Jarnuczak et al. 2021, *An integrated landscape of protein expression in human cancer*, Scientific Data (PMC8065022, open access)	The text, used in Parts 1 and 3	Europe PMC full-text XML

The loop in every part

1. **Write a prompt** in a text cell. A checker lists which parts of a good prompt are missing.
2. **Paste it into an AI assistant** and paste the code back into the marked cell.
3. **Run the check.** It tests the code against something the participant knows.
4. **Hunt the bug** in a ready-made AI answer that runs but is wrong.
5. **Fix the prompt** so it would have prevented the bug.

Without an assistant, participants skip step 2. Safety-net functions defined in Part 0 take over whenever a participant's own function fails, so one broken part never blocks the next.

What each part retrieves and how it is checked

Part	Minutes	Retrieves	Checked by
0	10	Nothing yet: setup and the prompt checklist	The prompt checker

Part	Minutes	Retrieves	Checked by
1	20	The full text of PMC8065022	Length, three known numbers, a bad ID
2	25	Tables S1 and S2 as data frames	Row and column counts, ERBB2 in breast lines
3	20	Four facts from the text, each with a quote	The quote is really in the paper
4	13	The shared table, protein plus RNA, copy number, mutation	No duplicates, and protein correlates with RNA
5	2	Saves the table	-

The notebook comes in a participant version, where the code cells are placeholders, and an instructor version with reference solutions. The reference solutions are what "a good AI answer" looks like.

Part 0: Setup and the prompt checklist (10 min)

Aim. Get the tools ready and teach what a good prompt for code contains, so that every later part starts from a clear request.

What the cells do

1. **Install and download the class file.** One cell installs `openpyxl` (reads Excel), `requests`, `pandas`, `scipy` and `pypdf` (reads PDFs). It then defines `get_class_file(name)`, which downloads a small data file from the course repository if it is not already in the session, rejects an answer that is really a web page, and prints a clear note if the download fails. It is called once for `ccle_panel.csv`.
2. **Imports and the prompt checker.** `grade_prompt(prompt, required)` searches a participant's prompt for patterns and prints which parts it found. `COMMON` holds five parts every prompt needs: a function signature, a return value, libraries, edge cases, and a self-check. Each part also adds its own (for Part 1: the Europe PMC source and "do not truncate"). It is pattern matching, not understanding, and the notebook says so.

3. **The safety net.** A collapsed block of reference functions: `get_paper`, `get_supplements`, `clean_table`, `build_dataset`. Participants are told not to read it before trying. It is used automatically when their own function raises an error.

What a good prompt contains

Part	What to write	Example
Goal	One sentence on the task	Get the full text of an open-access paper
Inputs	The exact source	The Europe PMC REST API, using the PMC ID
Function	Name, arguments, return value	`def get_paper(pmcid) -> dict` with keys `title`, `sections`, `text`
Libraries	Which to use	`requests` and `xml.etree`
Edge cases	What happens when things go wrong	Raise a clear error for an ID with no full text
Checks	A self-test and the assumptions made	Say what you assumed about the file layout

What participants do

Nothing is written in Part 0 except reading. They run the cells and see how the checker scores a vague prompt (a typical result is 0 of 7 parts found, with the note that fewer than 25 words usually leaves the AI guessing).

How to know it worked

`helpers ready` and `safety net ready` are printed. If `ccle_panel.csv` downloads, the output says `downloaded ccle_panel.csv`. If not, it prints a note and Part 4 will run with protein only.

What can go wrong

- `pip` install is slow when many people start at once. Ask for Part 0 to be run the day before.
- The class-file download fails if the repository files are not in `data/` yet, or the branch is not `main`.

Part 1: Get the paper (20 min)

Aim. Write a function `get_paper(pmcid)` that takes a PubMed Central ID and returns the paper's title, sections and full text. Participants learn that retrieval has traps and that a good function says what went wrong instead of returning something empty.

The task in detail

- **Input:** a PubMed Central ID such as `PMC8065022`.
- **Output:** a dictionary with keys `title`, `abstract`, `sections` (a list of heading and text pairs), `text` (the whole paper as one string) and `source` (where it came from).
- **Source:** Europe PMC, a free service. Its address is `https://www.ebi.ac.uk/europepmc/webservices/rest/<PMCID>/fullTextXML`.

Step 1a: a weak prompt and a stronger one

The weak prompt is "Write python to download the Jarnuczak cancer proteomics paper." The stronger one names the function, the library (`requests`, `xml.etree.ElementTree`, `pypdf`), the three routes to try in order, the return keys, an error for a bad ID, and "keep the whole text". The AI cannot guess the source, the keys, whether to keep everything, or the error behaviour, and that is why the second prompt wins.

Step 1b: what the reference code does

1. Check the ID matches `PMC` followed by digits, and raise a `ValueError` naming the ID if not.
2. **Route 1, XML.** Request the full-text XML. If the answer is HTTP 200 and starts with `<`, parse it: take the title, the abstract, and for each `<sec>` its title and all paragraphs. Each element's text is flattened, because XML hides text inside nested tags.
3. **Route 2, PDF.** If route 1 fails, download the PDF from Europe PMC and take the text out with `pypdf`.
4. **Route 3, a file.** If the PDF also fails and a file `paper.txt` exists in Colab, use its text.
5. If every route fails, raise a `RuntimeError` that lists the HTTP code of each route and tells the person to save the text as `paper.txt`.

This chain exists because the first Colab runs showed Europe PMC returning HTTP 500 for the Nusinow paper on both the XML and PDF routes.

Step 1c: the checks (`verify_part1`)

Check	Why
The keys `title` , `sections` , `text` exist	The contract of the function
The title is longer than 10 characters	A real title was found
At least one section	The structure was kept
The text has at least 20,000 characters	A paper is tens of thousands of characters; short means cut
The strings `191` , `0.58` and `7,171` are in the text	Numbers known from the paper: the abstract says 191 cancer cell lines and a median correlation of 0.58, and the results say 7,171 MS runs
`get_paper("not-an-id")` must raise an error	A bad input must fail loudly

Real result: title `An integrated landscape of protein expression in human cancer` , 16 sections, 47,124 characters, source `Europe PMC XML` , bad ID raised `ValueError` , PASS.

Step 1d: hunt the bug

`get_paper_flawed` looks fine and runs: it takes the XML text and cuts it with `[:5000]` ("keep it manageable"). It returns the right title and a text of 5,000 characters, one section, and the check prints `FAIL: the text is very short` and `FAIL: '7,171' is in the paper but not in your text` . The bad ID raises an XML `ParseError` rather than a clear message. The questions: which line cut the text, and what happens when you later ask "how many cell lines were profiled?" (the model answers from missing text, or guesses). And what happens downstream when a bad ID never fails.

Step 1e: fix the prompt

Participants add the sentence that would have prevented the bug (for example "return the whole text, do not truncate; raise an error for an invalid ID") and grade it again.

What participants do

Write a prompt, paste the AI's code over the placeholder, run the check, run the flawed demo, rewrite the prompt. Without an assistant they run the safety-net function.

What can go wrong

- Europe PMC can answer HTTP 500 for a paper. The chain falls back to the PDF and to `paper.txt`.
- If the source is the PDF or the file, the text has no section structure (one section called `full text`), and Part 3's "results section" is then the whole text.

Part 2: Supplementary tables (25 min)

Aim. Get the protein measurements. The paper points to its data, and the data live somewhere else: that is common. Participants write two functions and learn how messy real tables are.

The two tables

Table	What it holds	File used
S1	The cell lines: one row per cell line with name, CCLE code, tissue of origin and other metadata	`Table_S1_Sample_Information.xlsx`
S2	The proteins: one row per protein, one column per sample (a cell line in a mass-spec batch)	`protein_quant_current_normalized.csv.gz`, the lab's compressed version of the normalized table

Both come from `gygi.hms.harvard.edu/data/ccle/`. The lab updates the files, so the current S2 can differ slightly from the 2020 paper.

The two functions

`get_supplements(urls)`: input a dictionary of label to URL; output the list of local file paths. The reference code creates a `supp` folder, skips a file already downloaded, calls `raise_for_status()`, and refuses an answer whose content type is `text/html` (a web

page instead of data).

`clean_table(raw)`: input a sheet read with no header (so the first rows can be anything); output a proper data frame. The reference code:

1. drops completely empty rows and columns;
2. finds the **header row**: the first of the first 15 rows with at least half of its cells filled;
3. names the columns from that row and makes repeated names unique (`name`, `name.2`);
4. converts a column to numbers when at least 80% of its filled cells are numeric;
5. **keeps missing values**, because in proteomics a missing value means "not detected".

Step 2c: download, look, choose

One cell downloads the files, reads every Excel sheet and the CSV with `header=None`, and prints a catalogue of every table. It then guesses: S2 is the **widest** table, S1 is the **tallest** of the others with at most 80 columns.

Real catalogue:

Table	Rows	Columns
`Table_S1_Sample_Information.xlsx :: Guide`	9	2
`Table_S1_Sample_Information.xlsx :: Sample_Information`	389	6
`protein_quant_current_normalized.csv.gz`	12,756	426

The guess was correct. Participants check it by eye and can override `S1_KEY` and `S2_KEY`.

Step 2d: the checks (`verify_part2`)

Check	Expected
S1 rows	About 375 (within 30), a few more if replicates are listed
S2 proteins	At least 5,000 (so rows with any missing value were not dropped)

Check	Expected
Numeric sample columns	At least 300, so numbers are not stored as text. Only `_TenPx` columns count, so `Group_ID` and peptide counts are ignored
Missing values	Some must remain
ERBB2	Present, and its top three samples should include a **breast** line (HER2 is high in breast cancer lines)

Real result: S1 388 rows x 6, S2 12,755 x 426, PASS.

Step 2e: hunt the bug

The flawed answer runs without error. It takes the first row as the header (no search for the real header row), then calls dropna() to "remove incomplete rows", and never converts numbers. The check shows S1 shrinking to 10 rows, S2 to 5,150 proteins, and 0 numeric sample columns. Questions: why does a protein missing in one sample get thrown away; what would a model learn if proteins that are not detected in many lines were silently removed (the survivors are the abundant, easy ones); which sentence in the prompt would have stopped it ("keep missing values, do not drop rows").

What participants do

Write the prompt, paste the two functions, run the catalogue and check, run the bug demo, fix the prompt.

What can go wrong

- Reading the S2 file as Excel is slow; the notebook uses the CSV.
- The wrong table guessed as S1 or S2: set the keys by hand.
- The gene column is found by `find_gene_col`, which looks for a column named like `Gene_Symbol`.

Part 3: Facts from the text, with evidence (20 min)

Aim. Tables are the easy half. The text holds facts that are in no table. An LLM can read a section and answer in a structured form, but it can also invent a number or invent the sentence that supports it. The rule of this part: every extracted value must come with a

quoted sentence, and the quote must be a verbatim sentence from the paper. A quote that is not in the paper is the signature of a hallucination, and one line of code can check it.

Step 3a: ask the assistant

1. A cell prints the abstract and the Results section of the paper (19,652 characters in the real run) for participants to copy.
2. They paste that text into any AI assistant and ask for **four facts as JSON**. Each item has three keys: `fact` (the exact name), `value`, and `evidence` (a sentence copied from the text).
3. The four facts and what the paper says:

`fact`	Expected `value`	Where it appears
number of cancer cell lines	191	Abstract: "... 191 cancer cell lines and 246 clinical tumour samples ..."
number of MS runs reanalysed	7,171	Results: "... re-annotated 7,171 MS runs ..."
number of lineages	13	Abstract: "... across 13 lineages"
median protein-RNA correlation across cell lines	0.58	Abstract: "... median correlation across cell lines to be 0.58"

The prompt checker looks for five parts: JSON output, an evidence quote, a no-guessing rule, the four facts, and a format example.

Step 3b: paste and verify

Participants paste the JSON between triple quotes. If they leave the placeholder, the **participant** notebook uses a pre-made flawed answer, so the check can still be seen working. The **instructor** notebook uses a correct reference answer (all four verified).

`verify_facts` runs three tests per fact, after normalising quotes, dashes and spaces:

Test	Meaning
Quote is verbatim in the paper	The quote has more than 15 characters and appears in the paper's text
Value is in the quote	The number the model reported is in the sentence it quoted
Value matches paper	The value equals the number we know is right

A table shows the three results per fact and the notebook prints either `ALL FACTS VERIFIED` or how many facts failed.

Step 3c: what went wrong

The pre-made flawed answer fails three different ways, and each one looks reasonable to a human:

- a **paraphrased quote**: the idea is in the paper but the sentence is not ("more than seven thousand raw files from eleven studies");
- a **rounded value with an invented supporting sentence**: the correlation 0.6 instead of 0.58, with a sentence picked from elsewhere;
- a **real sentence from the wrong place**: a true sentence that does not contain the value.

Only the verbatim test catches them. Participants then rewrite the prompt so the assistant must copy sentences exactly and answer `null` when the text does not say, ask again, and re-run the check.

What participants do

Copy the text, ask the assistant, paste the JSON, run the verification, read which facts failed and why, rewrite the prompt.

What can go wrong

- The assistant adds extra keys or writes a trailing comment: `json.loads` fails. Ask it for JSON only.
- The assistant rewrites a quote with different quote marks or dashes. The normalisation handles common cases; others count as failures, which is correct.

- If the paper came from the PDF route, the text may have broken line joins and a true quote can fail to match.

Part 4: Build the shared dataset (13 min)

Aim. Everything retrieved so far becomes one table that the AlphaGenome, folding and final notebooks use. Participants meet the silent errors that appear when layers from different sources are combined.

The table

One row per (cell line, gene).

Column	Layer	Source
`cell_line`	key	CCLE name, for example `AU565_BREAST`
`gene`	key	Gene symbol
`protein`	protein	Table S2 (this notebook)
`rna_log2tpm`	RNA	`ccle_panel.csv`
`copy_number`	DNA	`ccle_panel.csv`
`mutation`	DNA	`ccle_panel.csv`, protein change such as `p.R175H`, blank if none

Only a panel of 20 cancer genes is kept (`PANEL`), not all 12,000 proteins. ESR1 has no protein in this data, so 19 appear.

The function `build_dataset(S2, gene_col, panel_genes, panel_file)`

The reference code does, in this order:

1. **Pick the sample columns.** The numeric columns of S2 that end in `_TenPx` and a number (for example `AU565_BREAST_TenPx05`). `Group_ID` and the peptide-count columns are numeric too and are excluded.
2. **Melt** the table from wide (one column per sample) to long (one row per protein and sample).

3. **Strip the batch suffix.** `AU565_BREAST_TenPx05` becomes `AU565_BREAST`. The suffix names the mass-spec batch, not the cell line, and left in it would stop every later merge from matching.
4. **Drop missing values, then average replicates.** A cell line measured in several batches gets one value per gene: the mean.
5. **Keep the panel genes** only.
6. **Merge** with `ccle_panel.csv` on (`cell_line`, `gene`) with an inner join. If the file is missing, print a note and return protein only.

Step 4b: the checks (`verify_part4`)

Check	Failing means
No duplicate (cell line, gene) rows	Replicates were not averaged
No `TenPx` in cell-line names	The batch suffix was not removed
Fewer than 20% of protein values equal to 0	Missing values were filled with zero
At least 10 panel genes	The gene filter went wrong
Mean protein-vs-RNA Spearman across genes above 0.2	Cell-line names did not match across layers

The last check is the strongest, because it tests the biology, not the code. Nusinow et al. report about 0.48 over all genes and Jarnuczak et al. a median of 0.58 across cell lines. If rows were shifted, matched to the wrong lines or the wrong genes, the correlation falls to about 0.

Real result: 6,106 rows, 19 genes, 371 cell lines, mean Spearman 0.45, PASS.

Step 4c: hunt the bug

The flawed answer runs and gives a table of the right shape. It uses all float columns (peptide counts included), melts the sample names into `cell_line` without removing the suffix, fills missing values with 0 ("avoid NaN problems"), and merges with `how="left"`. The suffixes make the merge find nothing, and the left join hides that: the RNA columns are simply empty. The check reports duplicate rows, the batch suffix, and a protein-RNA correlation that is `nan`. Real output: 7,938 rows, 378 "cell lines", FAIL on duplicates and the suffix.

The questions: which bugs does the check name; why does the merge silently find nothing and why does `how="left"` hide it; what would a model trained on this table learn (it would run, and would learn nothing about biology).

Step 4d: save the file

The table is written to `ccle_multiomics_panel.csv`, shown (first 8 rows), and offered as a download in Colab.

What participants do

Write a prompt describing S2's layout, the sample-name suffix, the merge keys and the missing-value rule; paste the code; run the check; run the bug demo; rewrite the prompt.

What can go wrong

- Without `ccle_panel.csv`, the table has protein only (`columns: cell_line, gene, protein`), and the biology check is skipped with a note.
- Only 371 of the 375 cell lines appear. Four have no match in the DepMap name table; they are harmless.
- PGR (about 36 lines) and PALB2 (about 160) have few protein measurements, so they are poor genes to use as examples.

Part 5, the files produced, and common problems

Part 5: wrap-up (2 min)

What participants now have, all retrieved by code from published sources and checked: the full text with verified quotes, the proteome of about 375 cancer cell lines, and `ccle_multiomics_panel.csv`. The notebook ends with five checking habits and a table that pairs each bug with the sentence that would have prevented it.

Bug met	Part	The prompt sentence that prevents it
Text silently cut at 5,000 characters	1	"Return the whole text, do not truncate"
Incomplete rows dropped	2	"Keep missing values; do not drop rows"
Invented or paraphrased quotes	3	"Copy sentences exactly; answer null if the text does not say"

Bug met	Part	The prompt sentence that prevents it
Zero fill, batch suffix kept, duplicates, wrong merge	4	"Strip the batch suffix, average replicates, keep NaN, use an inner join"

The habits: test against something you know; test the edge case; make claims carry evidence that code can verify; check a biological expectation; never accept code because it ran.

Files this notebook produces or uses

File	Role
`ccle_panel.csv`	Input to Part 4, downloaded from the course repository
`supp/` folder	The two downloaded supplementary files
`paper.txt`	Optional manual fallback for Part 1
`ccle_multiomics_panel.csv`	The output, used by the later notebooks

Before the workshop (instructor)

1. Download from DepMap (release 26Q3): `Model.csv`, `OmicsExpressionTPMLogp1HumanProteinCodingGenes.csv`, `OmicsCNGeneWGS.csv` and `OmicsSomaticMutations.csv`.
2. Run the last cell of the instructor notebook, `prepare_ccle_panel`. It keeps one default entry per cell line, reads only the panel genes, and writes `ccle_panel.csv` (34,900 rows) and `ccle_mutations.csv` (3,979 rows).
3. Run this notebook once with `ccle_panel.csv` present and keep the `ccle_multiomics_panel.csv` it saves.
4. Put the three files in `data/` of the course repository.

Common problems

Symptom	Cause	Fix
`Could not get the text of PMC...`	Europe PMC failed on every route	Save the paper's text as `paper.txt`, upload it, run again
The check says the text is short	A route returned only part of the text	Use the XML route, or `paper.txt`
`ccle_panel.csv not found` note	The file is not in the repository or not downloaded	Upload it, or continue with protein only
The wrong table is picked as S1 or S2	The guess uses width and height only	Set `S1_KEY` and `S2_KEY` by hand
JSON cannot be parsed in Part 3	The assistant added text around it	Ask for JSON only, no comments
Biology check skipped	No RNA column	Provide `ccle_panel.csv`

AlphaGenome and TP53 mutations

AlphaGenome in one paragraph

AlphaGenome (Google DeepMind) is a model that reads up to 1 million bases of DNA and predicts many molecular readouts: gene expression, splicing, chromatin accessibility. Participants do not run the model. They call it through a web API and get tracks of numbers back. Using the DNA layer of the CCLE cell lines (the somatic mutations), they ask what each TP53 mutation should do to TP53's RNA, then compare with the RNA actually measured in the cell lines that carry the mutation. It takes about 60 minutes, needs internet and a free API key, and no GPU.

The question and the prediction made in advance

A mutation that breaks a splice site, or creates a stop codon, should lower the gene's RNA (the cell destroys faulty transcripts). A silent change should do nothing. A model that reads only DNA sequence may see the first (splicing) and may not see the second (the cell's quality control, called nonsense-mediated decay). That gap is the lesson.

Words used in this notebook

Word	Meaning
Variant	One change in the DNA, defined by chromosome, position, reference base and new base
hg38	The version (build) of the human reference genome. Positions differ between builds
SNV	Single-nucleotide variant: one base replaced by one base
Splice site	The boundary between an exon and an intron, where the cell cuts and joins the message
Nonsense	A change that creates a stop codon and cuts the protein short
Missense	A change that swaps one amino acid for another

Word	Meaning
Silent (synonymous)	A DNA change that leaves the amino acid unchanged
Track	One predicted signal along the DNA (for example RNA-seq in one tissue)
Variant scorer	A recipe that turns the model's reference and alternate predictions into one number

The five parts

Part	Minutes	What happens
0 Setup	8	Install, API key, terms of use, connection test
1 The TP53 locus	10	Load the gene's location and predict an RNA track
2 Variants from the CCLE table	20	Build variants correctly and check them against the genome
3 Score and compare	16	Predicted effect against measured RNA
4 Pitfalls and open time	6	Genome build, strand, reference allele; challenges

Each part follows: predict, try (a TODO), check cell, debrief.

Learning outcomes

1. Connect to the AlphaGenome API and predict RNA-seq for a gene locus.
2. Build variants correctly from a data table: right position, right reference base, right genome build.
3. Score variants and read the tidy results table.
4. Compare predicted effects with measured RNA and say what the model can and cannot see.
5. Name three ways a variant-effect pipeline silently goes wrong: genome build, strand, reference allele.

Part 0: Setup (8 min)

Aim. Connect to AlphaGenome safely and know the terms of use before sending anything.

Before the session

Each participant requests a free API key at `alphagenome.google/api` and, in Colab, opens the key icon in the left sidebar (Secrets), adds a secret named exactly `ALPHA_GENOME_API_KEY`, and switches on Notebook access. If they skip this, the notebook asks them to paste the key instead, hidden as they type. They must never save a notebook with a key typed into a cell.

Terms of use (read aloud)

The API is free for non-commercial use. Outputs must not be used to train other machine-learning models and must not be used for clinical decisions. Rates vary with demand, so requests stay small.

What the cells do

1. **Install** `alphagenome`, `pyliftover` and `requests`.
2. **Imports.** NumPy, pandas, matplotlib, `requests`, `scipy.stats.spearmanr`, and the AlphaGenome modules (`colab_utils`, `gene_annotation`, `genome`, `transcript`, `dna_client`, `variant_scorers`, `plot_components`).
3. **`get_key()`** reads the Colab secret; if that fails it asks for the key with `getpass`, and raises a clear error if it is empty.
4. **`connect(key)`** creates the client, then sends one tiny request (a 16,384-base sequence of repeated ACGT asking for RNA-seq). The client does not check a key when it is created, so this test is how an invalid key is found. An invalid key produces a message listing three likely causes: the whole key was not copied, the secret's value is the secret's name, or the key was not issued by the API page.
5. **Print** `connected to AlphaGenome` and the supported input lengths: 16 kb, 100 kb, 500 kb and 1 Mb.

How to know it worked

`connected to AlphaGenome` and the list of four sequence lengths.

What can go wrong

Symptom	Fix
`API key not valid`	Re-copy the key into the secret's Value box, with no quotes or spaces
The notebook asks for a key	The secret name is wrong, or Notebook access is off; or paste the key
A note about the test request failing for another reason	A temporary service problem; the notebook continues

---

Part 1: The TP53 locus (10 min)

Aim. See what the model predicts for a whole gene before changing anything, and learn that gene strand matters.

What the cells do

1. **Load the gene annotation.** The notebook reads DeepMind's copy of the GENCODE v46 annotation (hg38) as a table, keeps protein-coding genes and one canonical transcript per gene (the MANE Select transcript), and builds a transcript extractor.
2. **Find TP53.** `gene_annotation.get_gene_interval` returns the interval `chr17:7661778-7687546:-`. The gene is 25,768 bases long on **chromosome 17**, on the **minus strand**.
3. **Predict.** The interval is widened to 1 Mb (the model's longest input) and the notebook requests RNA-seq predictions in one tissue, ontology term `UBERON:0001114` (right liver lobe, used in the AlphaGenome tutorial). The call takes a few seconds.
4. **Plot.** The transcript with its exons is drawn above the predicted RNA tracks, zoomed to the gene with a margin.

Input and output

Input: a gene name and a tissue term. Output: a predicted RNA-seq signal along 1 Mb, two strand tracks, shown next to the exons. There is nothing to code in this part.

What participants do

Write a guess about the strand first (it is written in the notebook as a prompt), run the cells, and answer the debrief: where does the signal sit relative to the exons; which strand track carries it, and does that match TP53's strand? A prediction is a model's reading of the sequence, not an experiment.

What can go wrong

The annotation file is large and the first load takes a minute. If the API is busy the prediction may need a re-run.

Part 2: Variants from the CCLE table (20 min)

Aim. Turn rows of a mutation table into correctly built AlphaGenome variants, and check them against the real genome before scoring anything.

The input table: `ccle_mutations.csv`

One row per mutation per cell line, hg38, 1-based positions. Columns used: `cell_line`, `gene`, `chrom`, `pos`, `ref`, `alt`, `variant_info` (the effect type, in Ensembl's words), plus `protein_change` and `variant_type`.

Example row: `SOME_LINE_LUNG, TP53, chr17, 7675088, C, T, missense_variant, p.R175H`.

Step 2a: load and filter

1. `get_class_file` downloads `ccle_mutations.csv` and `ccle_panel.csv` if they are not in the session.
2. The notebook checks the required columns exist and keeps the rows where `gene == "TP53"`: **1,365 records in 1,218 cell lines** in the real run.
3. It prints the commonest `variant_info` values: `missense_variant` (870), `stop_gained` (155), `frameshift_variant` (142), `splice_acceptor_variant` (50), `splice_donor_variant` (46), and others, some joined by `&`.

Step 2b: classes

Only single-base substitutions are kept (`ref` and `alt` are each one of A, C, G, T): **1,146 of the 1,365**. Insertions and deletions need another representation and are left out. Then `variant_class` sorts each into a class by the words in `variant_info` :

Class	Rule	Expected effect on RNA
splice	contains `splice_donor` or `splice_acceptor`	Lower: splicing breaks, transcript often degraded
nonsense	contains `stop_gained`	Lower: nonsense-mediated decay
missense	contains `missense`	No change
silent	synonymous, not near a splice site	No change
silent_near_splice	synonymous plus `splice_region_variant`	Unknown: that is the question

When several terms are joined by `&` , the strongest wins (`missense_variant&splice_region_variant` is missense). `splice_region_variant` alone is not counted as splice. Real counts of single-base TP53 variants: 873 missense, 162 nonsense, 94 splice, 17 synonymous. All 17 synonymous ones sit next to a splice site, so they land in `silent_near_splice` and `silent` is empty for TP53.

Step 2c: TODO 1, `to_variant(chrom, pos, ref, alt, name)`

It must return an AlphaGenome `genome.Variant` . Participants fix two things: the chromosome must start with `chr` without doubling it (`17` becomes `chr17` , `chr17` stays), and the position must be an integer while the bases are upper case. The check cell tests `to_variant("17", 100, "a", "G", "t")` .

Step 2d: the reference-base check

AlphaGenome does **not** stop you if you give the wrong reference base. A position off by one, or from another genome build, silently scores a different variant. So the notebook looks up the real base in the genome and compares it with the table's `ref` .

- `ref_base(chrom, pos)` asks the Ensembl REST service for the GRCh38 base at that position, and falls back to UCSC. It retries each twice, waits between tries, caches results, and raises a clear error if every attempt fails.
- The notebook picks up to `N_PER_CLASS` = 15 **distinct variants per class**, preferring variants carried by cell lines that have RNA data (each distinct variant is scored once even if several lines carry it).
- It builds the variants and prints the share whose reference base matches. **If fewer than 90% match, the notebook stops** (an `assert`): wrong build, or positions shifted by one. Variants that mismatch are excluded and named.

Real result (with 8 per class): 100% of 27 variants matched, in 38 seconds. Second run with 15 per class: 48 variants (15 splice, 15 nonsense, 15 missense, only 3 silent near splice), 100% matched, 135 seconds for the lookup and 5 seconds for scoring.

What it catches and what it does not

A table from an older paper in hg19 would fail for almost every row. A 0-based table fails for about 3 of 4 rows. It does **not** catch a variant on the wrong strand whose reference base happens to match.

Strand

The table's `ref` and `alt` are on the forward (+) strand of the genome, whatever the gene's strand. TP53 is read from the minus strand, so a `C>T` in the table is a `G>A` in the gene's own reading direction. AlphaGenome takes forward-strand variants and handles the gene's strand itself.

Part 3: Score and compare (16 min)

Aim. Ask AlphaGenome what each variant does, reduce thousands of numbers to one or two per variant, and compare with the RNA measured in the cell lines.

Step 3a: predict before scoring

Participants write a ranking in the notebook: which classes lower TP53 expression the most (splice, nonsense, missense, silent, silent near splice), and which classes AlphaGenome can see at all.

Step 3b: scoring

`score_variants` predicts both the reference and the alternate sequence for every variant and condenses the difference into a score with a **variant scorer**. Two recommended scorers are used:

Scorer	Measures
`SPLICE_SITES`	Change in predicted splice-site strength (donor and acceptor tracks)
`RNA_SEQ`	Change in predicted expression, per gene, in each of many tracks (cell types and tissues)

Each variant is given 131 kb of context (the model's 100 kb option) instead of 1 Mb, which is faster. The result goes through `tidy_scores`, which returns **one row per variant, gene and track**. In the real run 27 variants gave a table of 80,190 rows by 23 columns, because every gene in the window and every track counts. It took 2 seconds. The `variant_id` column is turned into a plain string such as `chr17:7675052:C>T`, because different client versions return text or an object.

Step 3c: one number per variant

- **Splice score**, `per_variant_splice`: for `SPLICE_SITES` rows of TP53, the largest absolute raw score over the tracks.
- **Expression change**, `per_variant_expression` (**TODO 2**): for `RNA_SEQ` rows of TP53, group by `variant_id` and take the **mean** of `raw_score` over all tracks. Negative means lower predicted expression. The check requires at least one result and no more rows than variants.

Step 3d: the measured side

For each variant, the notebook takes the cell lines that carry it and expresses their TP53 RNA as a **z-score** against the cell lines that have **no TP53 mutation record at all**.

- Reference group: 685 cell lines, mean log2 TPM 5.20, standard deviation 1.38.
- A z-score of -1.25 means the carrying lines are on average 1.25 standard deviations below the unmutated lines.

- Predictions and measurements are matched through the `variant_id` key, and the table `res` holds, per variant: splice score, expression change, class, number of carriers, and measured RNA z.

Step 3e: the pictures

Three panels: a box plot of the splice score by class, a box plot of the predicted expression change by class, and a scatter of predicted change (x) against measured RNA z (y), coloured by class. Below it, two Spearman correlations: predicted change against measured z, and splice score against how much lower the measured RNA is.

What the numbers meant (first real run, 8 per class)

Class	Splice score	Predicted change	Measured z
splice	0.99	-0.039	-1.25
nonsense	0.03	-0.012	-1.25
missense	0.02	+0.002	+0.49
silent (near splice)	0.70	0.000	-0.01

Spearman predicted against measured: +0.37 (p = 0.05, n = 27) in the first run and +0.36 (p = 0.01, n = 48) in the second. Second-run medians of the measured z: splice -0.87, nonsense -1.48, missense +0.63, silent near splice -0.01. AlphaGenome picks out splice variants by the splice score. For nonsense variants it predicts a small drop although the measured RNA is as low as for splice, which is the point of the debrief: nonsense-mediated decay is a cell process, not something the DNA sequence shows. Predicted changes (-0.04) and measured z-scores (-1.25) are on different scales, so only their order is compared.

What participants do

Write the ranking, run the scoring, complete TODO 2, read the table and the plots, and discuss: was the ranking right; why a DNA-only model cannot know about decay; how far to trust a comparison where each class has few variants, each seen in few lines of different tissues; and which step would break the rule against training other models on AlphaGenome's outputs.

What can go wrong

Symptom	Fix
`RESOURCE_EXHAUSTED` or quota errors	Wait a minute and re-run; lower `N_PER_CLASS`; many people share the service
No `RNA_SEQ` rows for TP53	Print `tidy['output_type'].unique()` and `tidy['gene_name'].unique()`
`variant_id` does not match	The id is built from chromosome, position, reference and alternate base; check how the client formats it

Part 4: Pitfalls and open time (6 min and beyond)

Aim. Make the three silent failures concrete.

Genome build

AlphaGenome and the CCLE table use hg38. Many older datasets, including most 2018-era papers, use hg19. The same base has a different coordinate in each. The notebook takes the first scored variant and lifts its position from hg38 to hg19 with `pyliftover`. In the real run `chr17:7675052` (hg38) became `chr17:7578370` (hg19), a shift of **96,682 bases**. A tool given hg19 coordinates but expecting hg38 would score the wrong place without complaint.

The questions for participants

1. What would happen if you passed hg19 coordinates to a tool that expects hg38?
2. Why is the reference-base check from Part 2 a good safety net, and what does it not catch? (A variant on the wrong strand whose base happens to match.)

The three pitfalls

Pitfall	What goes wrong	What catches it
Genome build	The position points to a different base	The reference-base check

Pitfall	What goes wrong	What catches it
Position convention	0-based versus 1-based shifts by one	The reference-base check (about 3 in 4 rows fail)
Strand	The base is written for the other strand	Not caught by the check; read the documentation

Challenges (pick one)

Challenge	What to do
A. More variants	Raise `N_PER_CLASS` to 25 (watch the quota). Does the picture sharpen?
B. Another gene	Set `GENE = "PTEN"` or `"KRAS"` at the top of Part 1 and re-run from there
C. Compare scorers	Repeat Part 3 with `SPLICE_SITE_USAGE` instead of `SPLICE_SITES`
D. Alternate allele	For one splice variant, score all three possible new bases. Do they differ?
E. Tissue effect	Colour the third plot by the tissue of each carrying line (the suffix of the cell-line name). Does measured RNA depend more on tissue than on variant class?

Expectations, timing and what to check

What participants are expected to do

- Arrive with an API key and the secret set up, and with Notebook access on.
- Write a prediction or ranking before each result and keep it, even when it is wrong.
- Read the reference-check output and refuse to score variants that fail it.
- Complete two short TODOs: `to_variant` (a few lines) and `per_variant_expression` (one `groupby` line).
- Respect the terms: non-commercial, no training on outputs, no clinical use.

What they should be able to say at the end

- What a variant is and why five facts (chromosome, position, reference, alternate, build) must be right.
- What AlphaGenome predicted well (splicing) and what it cannot see (nonsense-mediated decay).
- Why a comparison of a few variants in a few cell lines is weak evidence.
- The three silent pipeline failures.

Timing

Setup 8, locus 10, variants 20, scoring 16, pitfalls 6 minutes. The reference-base lookup takes about 1.5 minutes with 15 variants per class; the scoring takes seconds; the first annotation load takes about a minute.

Instructor checklist

1. Request keys and test one before the workshop; each participant uses their own key, so a quota problem is per person.
2. Confirm `ccle_mutations.csv` and `ccle_panel.csv` are in `data/` of the repository.
3. Run the notebook once. Expect about 15 variants for each of splice, nonsense and missense, and the `silent_near_splice` class with up to 17 TP53 variants.
4. Say the terms of use out loud.
5. If the API is slow for everyone, lower `N_PER_CLASS`, or share a saved copy of the `tidy` table.

Things to tell participants honestly

- The correlation between predicted and measured effects was +0.37 (p = 0.05) in the first run. It is a demonstration, not a finding.
- The measured RNA of a cell line depends on its tissue and every other mutation, not just the variant.
- The nonsense and splice classes both lower the measured RNA about equally. Only splice shows in the model's splice score.

Folding: protein structure (ESMFold) and mutations

Big picture: what this notebook is for

Aim. Test whether a structure predictor can "see" cancer mutations. ESMFold reads an amino-acid sequence and predicts a 3D structure. We fold TP53, KRAS, CDKN2A and PTEN fragments with and without real mutations seen in CCLE cell lines, then ask: do the structural changes ESMFold reports (shape change, confidence drop, burial) line up with what the cells actually show (less mutant protein than the RNA predicts)?

What ESMFold is. A protein language model (ESM-2) plus a folding head. Input: a string of amino-acid letters. Output: a PDB file with x, y, z coordinates for every atom and, in the B-factor column, **pLDDT**, the model's own confidence per residue (0 to 100; above 90 is very confident, below 50 is likely disordered or a guess).

Why it is a good lesson for biologists. The model was trained on natural proteins, so a single-letter change usually moves the prediction very little. The notebook teaches participants to (1) parse real data, (2) check it against a reference (UniProt), (3) handle raw structure files, (4) use a negative control, and (5) accept a modest, honest result.

What participants are expected to do. Four short TODOs (`parse_protein_change` , `parse_pdb` , `mutate` , `burial`), each a few lines with a test cell right after it, plus predictions written down before each result. Total about 75 minutes. No biology knowledge beyond "a mutation swaps one amino acid for another" is needed.

Data in. `ccle_multiomics_panel.csv` (downloaded automatically): 371 cell lines × 19 genes, with protein, RNA, copy number and a `mutation` text column. **Data out.** A per-variant table (RMSD, drop in pLDDT, burial, protein minus RNA), three Spearman correlations and a scatter plot.

Part 0: Setup (about 8 min)

What the cells do.

- Imports numpy, pandas, matplotlib, requests and scipy's `spearmanr` .
- Defines `get_class_file(name)` : if the file is not in the Colab folder, it downloads it from the course GitHub `data/` folder, rejects HTML error pages, and prints a plain note if the download fails.

- Sets `FOLD_MODE = "api"` (fold through the ESM Atlas web service, no GPU needed) or `"local"` (fold inside Colab on a GPU, slower to start but not dependent on the web service). A banner tells participants which GPU, if any, they have.
- Defines `GENES`, the four genes with their UniProt accession and the fragment (first, last residue) to fold: TP53 `P04637` residues 94-312 (the DNA-binding core), KRAS `P01116` 1-189, CDKN2A `P42771` 1-156, PTEN `P60484` 1-353. Fragments keep the folding fast and avoid the disordered tails that ESMFold cannot predict well.
- Reads the CCLE table and prints its size, and raises a clear error if the `mutation` column is missing (that happens if the class file from the data retrieval notebook was not used).

Input / output. In: nothing from the participant except running the cell. Out: `371 cell lines, 19 genes` and a `folds/` folder where every folded structure is cached by a hash of its sequence, so re-running never folds the same sequence twice.

Participant action. Pick the folding mode (default `api`) and run. If the web service is down, switch to `local` and change the Colab runtime to GPU.

Check. The printed counts match the expected 371 lines and 19 genes.

Part 1: Mutations and sequences (about 12 min)

Aim. Turn the messy `mutation` text into clean `(reference, position, new)` triples that a folding model can use, and prove they are correct.

The text format. DepMap writes protein changes as `p.R175H` (arginine at position 175 becomes histidine). Other types look different: `p.R213*` is a stop, `p.E285fs` a frameshift. A line can hold several changes joined by `;`, for example `p.R175H;p.G245S`.

TODO 1: `parse_protein_change(text)`. Participants complete one line: match each piece against the pattern `^p\.([A-Z])(\d+)([A-Z])$` and, if it matches, append `(ref, int(position), alt)`.

- Example in: `"p.R175H;p.G245S"`. Example out: `[("R",175,"H"), ("G",245,"S")]`.
- Example in: `"p.R213*"` or `"p.E285fs"`. Out: `[]`, because only missense changes can be folded.
- Empty text returns `[]`.

The UniProt reference check. `fetch_uniprot` downloads the canonical sequence of each protein. For every parsed change the notebook asks: is residue `pos` in UniProt really the letter `ref`? The result is the `ref_matches` column. This catches a table that uses another transcript or numbering.

- A safety assert stops the notebook if fewer than 90% match.
- Rows that do not match are dropped; the rest are kept as `muts`.
- A summary prints, per gene: carrier lines, distinct positions, fraction matching.

Real numbers. 691 gene-line pairs carry a mutation in the 19 genes; **292 missense carriers match UniProt** across the four genes. Protein lengths: TP53 393, KRAS 189, CDKN2A 156, PTEN 403.

What can go wrong. A regex that forgets `^` / `$` also matches `p.R175Hfs`; a position that is off by one (Python counts from 0) fails the UniProt check and the notebook says so.

Part 2: Folding and reading a structure (about 18 min)

Aim. Fold the TP53 core domain once, then learn to read the output file.

Cells.

- `fragment(gene)` slices the UniProt sequence by the residue range in `GENES`. For TP53 that is residues 94-312, 219 amino acids.
- `fold(seq)` hides the mode: `fold_api` posts the sequence to the ESM Atlas service, `fold_local` runs ESMFold on the Colab GPU. Both return PDB text and both cache it in `folds/`.
- The PDB text is not data yet. It is a column-formatted text file where each `ATOM` line is one atom.

TODO 2: `parse_pdb(text)`. Participants read the C-alpha (backbone) atoms only, one per residue, using fixed columns: x = `line[30:38]`, y = `line[38:46]`, z = `line[46:54]`, and the B-factor `line[60:66]` that ESMFold fills with pLDDT. The function returns an array of coordinates `[n,3]` and an array of pLDDT `[n]`. If the maximum is at most 1 it multiplies by 100, because some tools write 0-1.

Check. An assert compares the number of residues in the PDB (219) to the length of the sequence.

Plot. pLDDT along the TP53 core with dashed lines at 90, 70 and 50.

Real numbers. Mean pLDDT **92.3**; only 6 of 219 residues below 70. The core domain is predicted with high confidence, so structural comparisons on it are meaningful.

Participant discussion. What would a low-pLDDT stretch mean? (Disordered or unreliable; do not interpret shape differences there.)

What can go wrong. The web service can time out or return an error; the notebook tells participants to switch to `local`. Wrong column slices give `ValueError` when converting to float.

Part 3: Does folding see mutations? (about 17 min)

Aim. Fold the wild type and the three famous TP53 hotspots and measure how much the predicted structure changes.

Prediction first. Participants write down whether the shape and the local pLDDT will change a lot, a little or not at all.

Hotspots. R175H (helps hold the fold), R248Q and R273H (touch the DNA). Each is a single amino-acid swap.

TODO 3: `mutate(seq, pos, new_aa)`. One line: `seq[:pos-1] + new_aa + seq[pos:]`. The trap is Python's 0-based indexing against the biologists' 1-based numbering. Example: `mutate("MKT", 2, "A")` returns `"MAT"`.

Fragment numbering. The fragment starts at residue 94, so whole-protein residue 175 is position `175 - 94 + 1 = 82` in the fragment.

RMSD (`kabsch_rmsd`, provided). Root-mean-square deviation between wild-type and mutant C-alpha coordinates, after the best rotation and translation (Kabsch algorithm). Small is similar. Participants are told what it means, not asked to code it.

`compare_variant` folds both versions and returns the RMSD, mean pLDDT for each and the pLDDT in a 5-residue window around the site.

Negative control. The same 219 letters shuffled randomly: same composition, no biology. If the model gave it a high pLDDT, the confidence numbers would be meaningless.

Real numbers. RMSD R175H **0.37 A**, R248Q **0.12 A**, R273H **0.18 A**. Scrambled control mean pLDDT **32.9** against 92.3 for the real core. So the model clearly separates real from nonsense, but the hotspot mutants look almost identical to the wild type.

"What the model cannot see" discussion. ESMFold predicts the most likely fold of a sequence, not the energetics of destabilisation, so a mutation that unfolds the protein in the cell can still be folded confidently in the model. Participants are asked to say why this matters for variant interpretation.

What can go wrong. If participants pass the whole-protein position to `mutate` instead of the fragment position, the wrong residue changes; the assert inside `compare_variant` checks that the residue at that position is really the expected reference letter.

Part 4: Structure versus measured protein (about 13 min)

Aim. Compare what the model predicts with what the cells show.

The measured signal. For each gene, Take the cell lines with no mutation in that gene as the reference. Turn protein and RNA of a carrying line into z-scores (standard deviations from the reference). **`protein minus RNA`** = protein z minus RNA z. Negative: less protein than the RNA predicts, as if the mutant protein is degraded. Zero: protein follows RNA. Example: a line with RNA z = +0.2 and protein z = +2.4 has a signal of +2.2.

The structure features (one per variant): RMSD, **dpLDDT** (wild-type minus mutant pLDDT near the site) and **burial**.

TODO 4: `burial(xyz, radius=10)`. From the `[n,3]` coordinates, build the `[n,n]` distance matrix, count entries below 10 A in each row and subtract 1 so a residue does not count itself: `(d < radius).sum(axis=1) - 1`. More neighbours means more buried, and buried residues usually tolerate change worse.

Choosing which variants to fold. `PER_GENE = 4` variants per gene (the most frequent ones inside the folded fragment, carried alone, meaning the line has a single change in that gene so the signal is not mixed), and only those carried by at least `MIN_LINES = 3` cell lines, so each point rests on several cell lines rather than one. A per-gene table is printed so participants see how many variants each gene contributes.

Folding time. About 16 variants, each folded twice (wild type is cached): roughly 8 min on a GPU, 507 s in the real run.

Analysis. Spearman correlation between the measured signal and each of the three structural features, followed by a scatter plot.

Real numbers. In the second run only 9 variants passed MIN_LINES (4 TP53, 4 KRAS, 1 CDKN2A, none for PTEN) and folding took 11 s after the GPU fallback; the Spearman values there were RMSD -0.73 (p = 0.02), burial -0.27 (p = 0.48) and dpLDDT +0.18 (p =

0.65), and the RMSD result rests on one point, CDKN2A H83Y (RMSD 1.37 A, signal -0.79). First run: TP53 variants carry a signal of +1.9 to +3.5, KRAS +0.3 to +0.8 (note that TP53 mutants usually give *more* protein than RNA predicts, because mutant p53 is stabilised, the opposite of the simple degradation story). Spearman: RMSD **-0.55 (p = 0.03)**, burial **-0.44 (p = 0.09)**, dpLDDT **-0.36 (p = 0.17)**.

How to read it. With about 16 points these are suggestive, not proof; only RMSD reaches p < 0.05, and a large part of the pattern is a difference between genes (TP53 against KRAS). The correct take-home is cautious: the structure features carry some signal, and a model trained on them would need many more variants.

What can go wrong. A gene with fewer than 3 carrier lines for a variant is dropped by `MIN_LINES`; if a table has no candidates, lower it. Spearman on very few points gives wide p-values.

Part 5: Challenges, expectations and common problems (about 7 min)

Challenges A to E (optional, for fast groups): change `radius`, fold a different gene (add it to `GENES` with its UniProt id and fragment), use another structure feature, repeat with a different `PER_GENE`, or compare the correlation within a single gene.

What participants should leave with.

- A single amino-acid swap usually barely moves an ESMFold prediction (RMSD well under 1 A).
- pLDDT is a confidence score, not proof of correctness, and the scrambled control shows it can fail low.
- Raw files need parsing and validating before use; a reference check (UniProt) is cheap and catches numbering mistakes.
- A correlation from 16 points is a hypothesis, not a result.

Timing. Part 0: 8 min, Part 1: 12, Part 2: 18, Part 3: 17, Part 4: 13, Part 5: 7. About 75 minutes, with folding being the main variable.

Instructor tips.

- Run the folding cells for the first TP53 core *before* the session if the API is slow; the `folds/` cache is keyed by sequence.
- Keep `FOLD_MODE = "api"` as the default and have a GPU runtime as fallback. Pre-folded structures were deliberately not shipped.

- Do not promise a significant correlation; the real run gave one significant out of three, which is a good talking point about multiple testing and small samples.

Common problems.

- *Download failed:* the class file did not download; upload `ccle_multiomics_panel.csv` by hand.
- *UniProt assert fails:* the table uses another isoform; print a few rows and compare with the UniProt page.
- *Folding service error or timeout:* switch to `local` and set the runtime to GPU.
- *Residue mismatch assert in `compare_variant`:* the fragment offset was applied twice or not at all.
- *Out of memory in local mode:* restart the runtime and fold only the shorter genes (KRAS, CDKN2A).